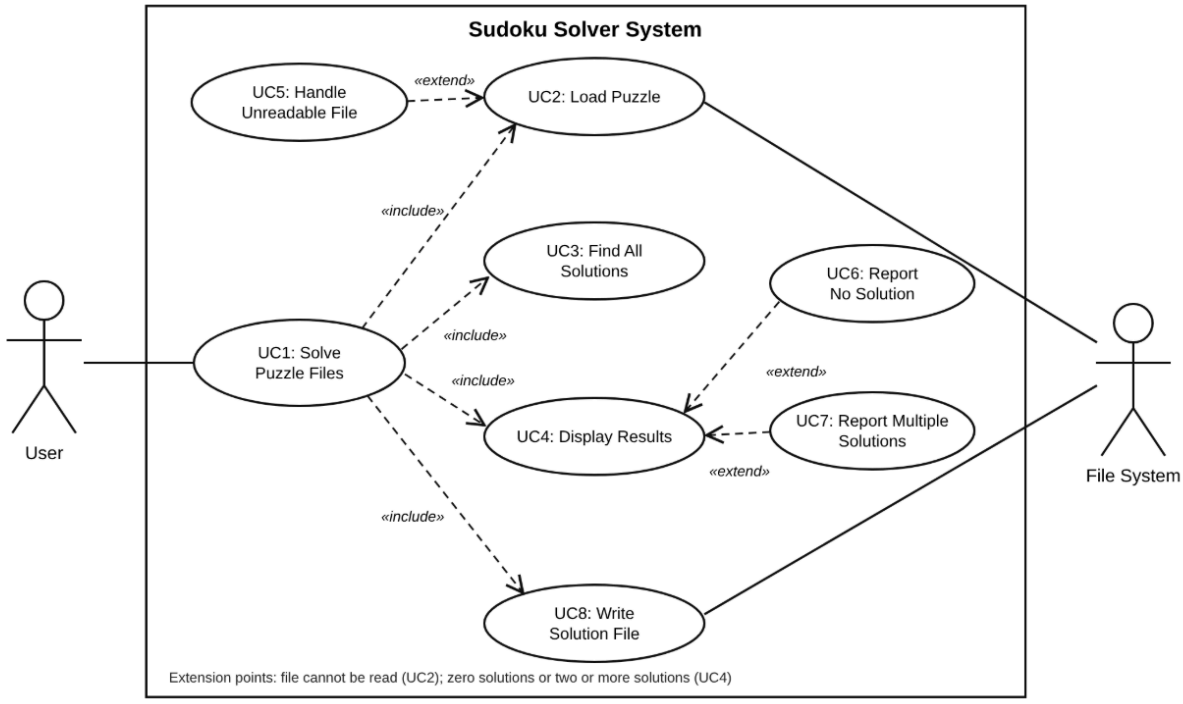


Made by Claude Opus 5.5 (Medium)

Use a GenAI of your choice to help you create a UML Use Case Diagram for this assignment with the level of detail given in the first UML Use Case Modeling lecture:

a. **Identify/discover a use case**

Solve Puzzle Files. The user runs the program to solve one or more Sudoku puzzle files and save each puzzle's solutions to a matching solutionN.txt file.

b. **Provide a brief description for the use case**

The user runs the Sudoku Solver and gives it puzzle files on the command line, or uses the five default files. For each file, the system reads and checks the puzzle, then prints it. It uses a recursive depth-first search with backtracking to find every solution, with no limit on how many. Each solution is printed on the screen and written to solutionN.txt as soon as it is found, where N is the puzzle number. The file uses the same format as the puzzle files and ends with "Solutions found: n". If there is no solution, the system prints "No solution found" and the file contains only "No solution found." If a puzzle file is missing or not a valid 9x9 puzzle, the system reports an error, writes no solution file, and moves on to the next file.

c. **Provide a step-by-step flow (outline) of the events for the use case**

Step 1: **The user starts the program, either with a list of puzzle files or with none.**

Step 2: **If no files are given, the system uses puzzle1.txt through puzzle5.txt.**

Step 3: **For each file:**

i. The system prints the file name.

ii. The system opens and reads the file (Load Puzzle).

iii. The system checks that the file has exactly 81 cells, each a digit 1–9 or an underscore (_).

iv. The system prints the puzzle as read (Display Results).

v. The system creates solutionN.txt, where N is the puzzle number (Write Solution File).

vi. The system checks that no given digits conflict.

vii. The system searches for every solution (Find All Solutions). Each solution is printed on the screen and written to solutionN.txt as it is found.

viii. The system prints and writes "Solutions found: n". If there are none, it prints "No solution found" and writes "No solution found." to the file (Report No Solution).

Step 4: After the last file, the system reports how many files could not be processed, if any, and the program ends.

Alternate flows:

- At step 3.ii, if the file can't be opened, the system prints an error, writes no solution file, and skips to the next file (Handle Unreadable File).
- At step 3.iii, if the file has a bad character or the wrong number of cells, the system prints an error, writes no solution file, and skips to the next file (Handle Unreadable File).
- At step 3.v, if solutionN.txt can't be created, the system prints an error and skips to the next file.
- At step 3.vi, if given digits conflict, the system prints "No solution found" with the reason, writes "No solution found." to the file, and skips the search.

d. **Provide a detailed description for the use case**

The Solve Puzzle Files use case begins when the User, a person running the program from the terminal, starts the Sudoku Solver. Before it starts, the program must be compiled and the puzzle files must be in the working directory or given by path. The user may list puzzle files on the command line; if none are given, the system uses puzzle1.txt through puzzle5.txt. For each file, the system prints the file name and loads the puzzle (Load Puzzle). It checks that the file has exactly 81 cells, each a digit from 1 to 9 or an underscore marking a blank, then prints the original puzzle (Display Results). Next, the system creates solutionN.txt, taking N from the puzzle file name, so puzzle3.txt produces solution3.txt (Write Solution File). It checks that no given digits conflict in any row, column, or 3x3 block, then runs a recursive depth-first search with backtracking (Find All Solutions). At each step, the search fills the empty cell with the fewest legal digits, tries each legal digit, recurses, and backtracks when it reaches a dead end. There is no limit on the number of solutions. Each time all 81 cells are filled, the system prints that solution with its number and writes it to solutionN.txt, with the grids in the file using the same format as the puzzle files. When the search ends, the system prints and writes "Solutions found: n" (Report Multiple Solutions when there is more than one). If there are none, it prints "No solution found" and writes the single line "No solution found." to the file (Report No Solution). Several alternate

flows can occur. If a puzzle file cannot be opened, or has an invalid character or the wrong number of cells, the system prints an error with the reason, writes no solution file, and moves on to the next file (Handle Unreadable File). If solutionN.txt cannot be created, the system prints an error and moves on. If the given digits conflict, the system prints “No solution found” with the reason and writes “No solution found.” to the file without searching. The use case ends when every file has been processed, and the system reports how many files, if any, could not be processed. The puzzle files themselves are not changed. The system assumes each puzzle is a standard 9x9 Sudoku with blanks marked by underscores, and it ignores whitespace between cells.